MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasile Brînză, FAF-242

Report

Laboratory Work No. 2
Operating Systems
Topic: Meet the OS You Will Build (xv6)

Checked by:
Reiţman Patricia, *university assistant*

Chişinău 2026

Table of Contents

1 INTRODUCTION....3
1.1 Aim of the work....3
1.2 The xv6 operating system....3
1.3 Working environment....3
2 PART 1. BUILD IT AND BOOT IT....4
2.1 Preparing the source tree....4
2.2 Compiling and booting....4
2.3 Leaving xv6....4
3 PART 2. USE XV6 AS THE UNIX IT IS....5
3.1 Running the built-in programs....5
3.2 Programs shipped with xv6....5
3.3 Features required for a pipe....5
3.4 Comparison with the Linux shell....5
4 PART 3. READ THE SOURCE....6
4.1 Reading a user program....6
4.2 System calls used by user/cat.c....6
4.3 Implementation of sys_read....6
4.4 Difference between kernel/ and user/....6
5 PART 4. WRITE YOUR FIRST XV6 PROGRAM....7
5.1 The program user/sleep.c....7
5.2 Registering the program in the Makefile....7
5.3 Rebuilding and testing....7
CONCLUSION....8
REFERENCES....9

1 INTRODUCTION

1.1 Aim of the work

In the first laboratory work the operating system was observed from the outside, on a real Linux machine. The purpose of this laboratory work is to move one step closer to the inside of an operating system: to build a small teaching OS from source, boot it, use it as the small Unix it is, read some of its code and write a first program that runs inside it.

The work is divided into four parts: building and booting xv6 (Part 1), using xv6 as a Unix system (Part 2), reading the source code (Part 3) and writing a first xv6 program (Part 4). For Parts 2 and 3 the answers to the “Observe” questions are given, and for Part 4 the new user program `sleep` is presented.

1.2 The xv6 operating system

xv6 is a small, real, bootable Unix operating system. It is a modern re-implementation of Unix Version 6 in about 9,000 lines of C, and it runs on the RISC-V architecture under the QEMU emulator. Because the whole system is small, it is possible to read both sides of the system: the programs that ask for services and the kernel code that answers.

The source tree is split into two main folders, and this split is the heart of how an operating system is organised:

- `kernel/` is the operating system itself. It runs in privileged mode and contains the scheduler, memory management, the file system and the system-call handlers.
- `user/` holds the ordinary programs that run on top of the kernel, such as `ls`, `cat`, `echo` and the shell.

1.3 Working environment

The work was done on the provided Linux virtual machine, where the RISC-V cross-compiler and QEMU are already installed. The course copy of xv6 is located in `~/xv6-riscv`; it can also be obtained from the public repository of the MIT PDOS group (see the references).

2 PART 1. BUILD IT AND BOOT IT

In this part the kernel and the user programs are compiled from source, and xv6 is started in the emulator.

2.1 Preparing the source tree

The xv6 directory was entered, or cloned first if it was not present on the machine:

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls
```

The listing of the directory shows, among other entries, the folders `kernel/` and `user/`, the `Makefile` and the `README` file.

2.2 Compiling and booting

A single command compiles the kernel and the user programs and then launches xv6 in the QEMU emulator:

```
make qemu
```

After a few seconds the kernel comes up and drops the user into a shell. The expected boot messages are:

```
xv6 kernel is booting

init: starting sh
$
```

Figure 2.1: Expected boot output of xv6

The message `init: starting sh` shows that the first user process, `init`, was started by the kernel and that it launched the shell. The `$` character is the prompt of the xv6 shell.

2.3 Leaving xv6

xv6 runs inside QEMU, so it is not left with a command of its own. To quit the emulator, `Ctrl-A` is pressed, released, and then `X` is pressed. This is the QEMU exit sequence and not a shortcut inside xv6.

3 PART 2. USE XV6 AS THE UNIX IT IS

At the xv6 prompt, the built-in programs were run. They feel familiar, because xv6 is the ancestor of the Linux system used in Lab 1.

3.1 Running the built-in programs

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c
$ wc README
$ usertests -q
```

The last command, `usertests -q`, is the kernel test suite; it is optional and takes a while to finish. The fourth command, `ls | grep c`, shows that pipes work in xv6 too.

3.2 Programs shipped with xv6

Three of the programs that xv6 ships with are:

- `ls`, which lists the files of a directory;
- `cat`, which prints the content of a file;
- `echo`, which prints its arguments.

3.3 Features required for a pipe

A pipe (`|`) was used inside xv6 in the command `ls | grep c`. Two operating system features must exist for it to work:

- **Process creation and scheduling.** The OS must be able to create and run two concurrent processes, the writer and the reader, using the `fork` and `exec` system calls.
- **Inter-process communication (IPC).** The OS must provide a shared in-kernel byte buffer, the `pipe`, which redirects the standard output of the writer process into the standard input of the reader process.

3.4 Comparison with the Linux shell

The xv6 shell provides the essential UNIX capabilities, such as command execution, piping and file redirection, in under 500 lines of C. However, it lacks the modern conveniences of the Linux Bash shell used in Lab 1, such as tab completion, command history, environment variables and job control.

4 PART 3. READ THE SOURCE

After quitting QEMU (`Ctrl-A`, then `X`), the source tree was opened in an editor. The code was read in three steps: a user program, the list of system calls, and the place where a system call is implemented in the kernel.

4.1 Reading a user program

Two small user programs were displayed with:

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
```

Both programs are tiny and have the same shape as the programs written in the C laboratory works: an `#include` section, a function that does the work and a `main` function.

4.2 System calls used by user/cat.c

In `user/cat.c`, the functions `read()` and `write()` are not ordinary functions: they are requests into the kernel. The system calls available to programs are declared in `user/user.h`, and their kernel handlers were searched with:

```
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

The system calls used by `user/cat.c` and the request that each one makes to the kernel are summarised in Table 1.

Table 1: System calls used by user/cat.c

System call	What it asks the kernel to do
`read()`	Fetch data bytes from an open file descriptor into a buffer in user memory.
`write()`	Output data bytes from a buffer to standard output (file descriptor 1).
`open()`	Open a file path and return an associated file descriptor.
`close()`	Release an active file descriptor.
`exit()`	Terminate the calling process and clean up its resources.

4.3 Implementation of sys_read

The handler `sys_read` is implemented in the kernel file `kernel/sysfile.c`, at line 69.

4.4 Difference between kernel/ and user/

The code in `kernel/` runs in privileged supervisor mode, with direct control over the physical RAM and the hardware devices. The code in `user/` runs in unprivileged user mode and must issue system calls whenever it needs a service from the kernel.

This is the main advantage of xv6 compared with Linux. On Linux, programs can only be seen from the outside; in xv6 both sides can be read: the program that asks (`user/`) and the kernel code that answers (`kernel/`).

5 PART 4. WRITE YOUR FIRST XV6 PROGRAM

In this part a new user program, `sleep`, is added to xv6. It pauses for a number of clock ticks by calling the `sleep` system call, which already exists in the kernel. This is the classic first exercise on xv6.

5.1 The program user/sleep.c

The program is written in the new file `user/sleep.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1])); // system call into the kernel
  exit(0);
}
```

The program works as follows:

- The check `argc != 2` verifies that exactly one argument (the number of ticks) was given. Otherwise, the usage message is printed to file descriptor 2 (the standard error) and the program exits with the status `1`, which signals an error.
- `atoi(argv[1])` converts the text argument into an integer number of ticks.
- `sleep(...)` is the system call into the kernel; it makes the process wait for the given number of clock ticks.
- `exit(0)` ends the program with the status `0`, which signals success.

5.2 Registering the program in the Makefile

The build must be told about the new program. In the `Makefile`, the `UPROGS` list was extended with a line for `sleep`. The indentation of the line above it must be copied exactly (it is a tab, not spaces), and the backslash at the end of the line must be kept:

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\    # <-- added line
        ...
```

5.3 Rebuilding and testing

xv6 was rebuilt and booted again, and the new program was tested inside it with two commands:

```
make qemu
$ sleep 10
$ sleep
```

- `sleep 10` should pause for ten ticks and then return to the prompt.
- `sleep` without an argument should print the usage message `usage: sleep <ticks>`.

CONCLUSION

In this laboratory work the xv6 teaching operating system was built from source and booted in the QEMU emulator. It was used as the small Unix it is, with its built-in programs `ls`, `cat` and `echo`, and with pipes between programs.

The pipe example showed that even a minimal Unix needs two core mechanisms: creation and scheduling of processes, and communication between them through a kernel buffer. Compared with the Linux shell from Lab 1, the xv6 shell offers the essential capabilities in under 500 lines of C, but without the modern conveniences such as history, tab completion or job control.

Reading the source code made the organisation of an operating system clear. The `user/` programs, such as `cat`, run in unprivileged mode and can only ask the kernel for services through system calls (`read`, `write`, `open`, `close`, `exit`), while the `kernel/` code, for example `sys_read` in `kernel/sysfile.c`, runs in privileged mode with direct control of memory and devices.

Finally, a first program, `sleep`, was written and registered in the build system. It validates its arguments and calls the `sleep` system call, which prepares the way for the next laboratory works, where the kernel side of xv6 will be modified as well.

REFERENCES

1. MIT PDOS, “xv6-riscv,” GitHub.com. [Online]. Available: https://github.com/mit-pdos/xv6-riscv

2. Wikipedia, “Xv6,” Wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/Xv6

3. QEMU Project, “QEMU – a generic and open source machine emulator and virtualizer,” qemu.org. [Online]. Available: https://www.qemu.org

4. Wikipedia, “System call,” Wikipedia.org. [Online]. Available: https://en.wikipedia.org/wiki/System_call